

IT8118: Advanced Programming

Group Project & Individual Reflection

Assessment Description

Semester:	Semester B, 2025-2026
Tutors:	Mr. Ghassan AlShajjar (Course Coordinator), Mr. Husain Naser, Dr. Ahmed Qusay
Learning Outcomes Covered:	• CILO 1: Implement web and data-driven applications that access and manipulate a database using modern frameworks and tools • CILO 2: Design custom database models and entity classes using object-oriented techniques • CILO 3: Manage a multi-layered application that uses structured components to access, manipulate, and expose data • CILO 4: Evaluate appropriate design and development concepts, approaches, and best practices for real-world programming problems
Weighting:	• Group Project – 30% • Reflection (Individual) – 10%
Deadlines:	• Group Project – 30th May 2026, 11:55 PM • Reflection (Individual) – Week 12 (24th – 28th May)
Notes:	<u>Extensions:</u> • Requests for extensions should be made 48 hours prior to the deadline and sent to the Course Coordinator. Extensions will only be approved for valid reasons with verifiable documentation. Extensions are for the whole group. You are only permitted a maximum of one extension per course per semester. <u>Late submissions:</u> • All late submissions (less than 3 days) will be capped at 60%. All late submissions (more than 3 days) will receive 0 grade.

Table of Contents

1. Project Overview.....	3
2. System Architecture	3
Project 1: ASP.NET Core Web API.....	3
Project 2: ASP.NET Core MVC Application	4
Project 3: Reporting Application.....	4
3. Project Briefs	7
Brief A: Healthcare Clinic Appointment & Resource System	7
Brief B: Property Leasing & Maintenance Platform	10
Brief C: Training & Certification Platform.....	12
4. Deliverables.....	14
5. Quality Standards	15
6. Marking Criteria.....	17
7. Group Work & Individual Accountability	18
8. Individual Reflection	19

1. Project Overview

In this project, your group must design and develop a complete web-based system for a real-world business scenario. The system will demonstrate your ability to architect a multi-layered application using modern .NET technologies, including a Web API backend, an ASP.NET Core MVC frontend, Entity Framework Core for data access, real-time functionality using SignalR, and deployment to Azure.

You will choose one of three project briefs provided below. Each brief describes a business domain and its core requirements at a high level. It is your responsibility to analyze the scenario, identify the necessary entities and relationships, design the database schema, and determine the full set of features your system needs to solve the business problem effectively.

2. System Architecture

Your submission must be structured as a single Visual Studio solution containing three projects that work together:

Project 1: ASP.NET Core Web API

This project serves as both the shared data layer and the API backend. It contains your domain entity classes, DbContext, and all Entity Framework Core configuration. It connects directly to the SQL Server database and exposes RESTful API endpoints.

As a shared data layer: The MVC application references this project to use the domain models and DbContext. This allows the MVC application to perform its core CRUD operations, authentication, and business logic via EF Core directly. This is the primary data access path for most features within the MVC application.

As a Web API: This project exposes RESTful endpoints secured with JWT authentication. These endpoints are consumed by two separate clients:

- **The MVC application** uses HttpClient to call the API for the **public tracking/lookup page**. This page allows unauthenticated users to look up information without logging in.
- **The Reporting Application** is a completely separate client that authenticates via JWT and retrieves all its data exclusively through the API. It has no direct database access.

Project 2: ASP.NET Core MVC Application

This is the main user-facing web application. It provides the full user interface with views, forms, navigation, and role-based access. For most features, the MVC application accesses the database through EF Core by referencing the shared domain models and DbContext from the API project.

The only feature in the MVC application that must consume the API via HttpClient is the **public tracking/lookup page**. All other features (CRUD operations, authentication, role-based workflows) use EF Core directly.

Project 3: Reporting Application

This is a separate application that simulates an internal reporting tool. It accesses the same data as the main system but does so entirely through the Web API. It does not have a project reference to the API project and has no direct database access.

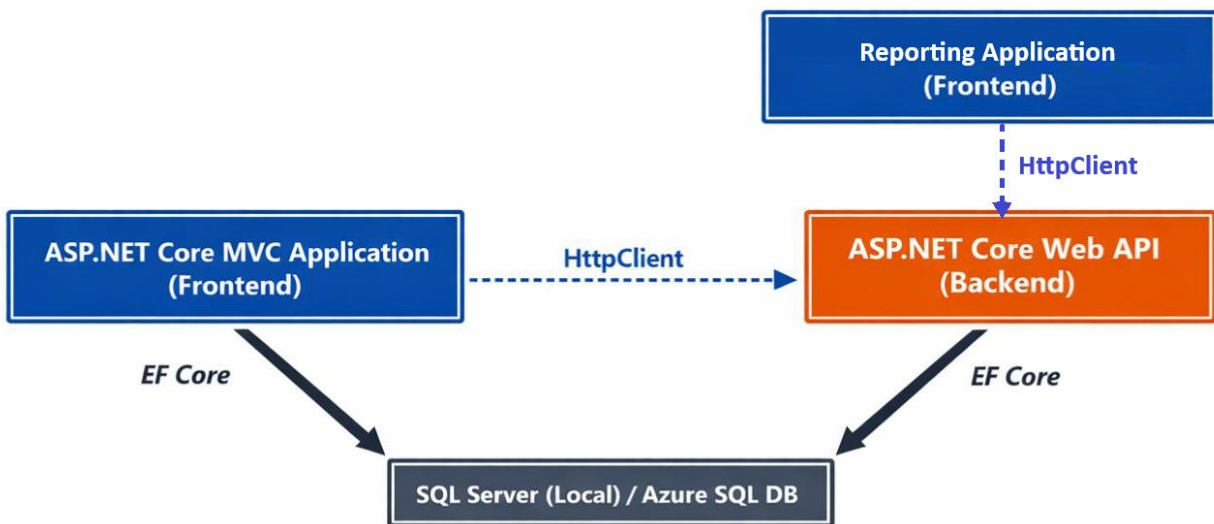
Technology choice: Your group may build the reporting application using ASP.NET Core MVC, Razor Pages, or Blazor Server.

The reporting application is expected to:

- Authenticate with the Web API using JWT to obtain a token before making any data requests
- Retrieve all displayed data by calling API endpoints via HttpClient

- Be restricted to the senior business role in your chosen brief (Clinic Manager, Property Manager, or Training Coordinator)
- Display operational reports relevant to the business scenario (see the reporting guidance under each brief)

Note: The reporting application is a read-only tool. It retrieves and displays data but does not create, update, or delete records.



API Requirement

You must expose a **minimum of 5 API endpoints** that cover at least two distinct functional areas of your chosen brief. These endpoints must follow REST conventions, using appropriate HTTP verbs, status codes, JSON responses, and are secured with JWT authentication where applicable.

Your project document must include a summary table of all API endpoints (route, HTTP method, purpose, authentication requirement).

SignalR Requirement

Your system must include **at least one real-time feature implemented using SignalR**. This should address a genuine need in your chosen business scenario (e.g., live notifications, real-time status boards, instant updates, chat sessions).

Azure Deployment Requirement

Your final submission must be deployed to **Microsoft Azure**. This includes deploying the MVC application, Web API, and Reporting Application to **Azure App Service** and the database to **Azure SQL Database**. Your deployed application must be functional and accessible via public URLs at the time of submission. The deployed URL(s) should be included in the project document.

Azure accounts will be provided to each group with limited usage credits. Use your credits wisely, avoid creating unnecessary resources, and delete any test deployments that are no longer needed. **Only one group member is required to perform the deployment as this is a group submission**, it is recommended that the group focuses on development and testing locally to avoid heavy credit usage.

3. Project Briefs

Choose one of the following three briefs. A maximum of 2 groups per class section may select the same brief. Brief selection will be coordinated and confirmed by the tutor.

Note on user roles: *None of the briefs include an “Administrator” role. In each scenario, the most senior business role carries elevated privileges including user management, system configuration, and full data access. Your system should still support role-based access control and user/role management, these responsibilities will belong to the senior business role user.*

Brief A: Healthcare Clinic Appointment & Resource System

Scenario

A medical clinic wants to replace its paper-based scheduling system with a web-based platform. The clinic provides a range of medical specialties and employs multiple doctors. Patients currently call in to book appointments, which may lead to double bookings, missed appointments, and limited visibility into doctor’s availability. The clinic management team requires a system that handles the appointment lifecycle from booking through completion, manages doctor schedules, tracks patient medical history, and provides operational visibility.

Roles

Patient, Doctor, Receptionist, Clinic Manager

Core Functional Requirements

The following areas describe the core business needs your system must address. This list is not exhaustive; you are expected to analyze the scenario and identify additional requirements, features, and edge cases that would make the system complete and usable.

1. **Doctor and specialization management.** Each doctor may specialize in multiple fields and has a defined availability schedule (working hours, days off, leave periods). The system should allow the clinic manager to manage doctor profiles/schedules and handle the impact on existing appointments when a doctor's availability changes.
2. **Appointment booking.** Patients or receptionists can book appointments by selecting a specialization, doctor, date, and time slot. The system must prevent double-booking based on doctor's availability.
3. **Appointment lifecycle management.** Each appointment follows a workflow: Requested → Confirmed → Checked-In → In Progress → Completed / Cancelled / Missed. The system should enforce valid transitions and handle the side effects of each status change.
4. **Patient medical records and visit history.** Each completed appointment generates a visit record that includes doctor's notes, diagnosis, and prescribed treatment. A patient's profile should show their full visit history across all doctors. Patients can view their own records; doctors can view records for their own patients.
5. **Prescription recording.** Doctors can record prescriptions linked to a completed appointment. Prescriptions should be viewable by the patient and the prescribing doctor.
6. **Notifications.** Patients and doctors should receive in-system notifications when relevant appointment events occur.

Public Appointment Lookup

A public web page where a patient can enter their CPR number and a patient reference number (without logging in), in which they will be able to view their upcoming appointments and recent visit summary. The page must retrieve the appointment details by calling a Web API using HttpClient. **This feature supports patients who do not have an account or are unable to access it.**

Real-Time Appointment Tracking

A live display that updates in real time when appointment statuses change, such as a waiting room screen or receptionist view showing live queue and appointment progress.

Reporting Application Guidance

The reporting application for this brief should provide the Clinic Manager with operational reports. Consider reports such as appointment statistics over a given period, doctor utilization and workload distribution, cancellation and missed appointment rates, or any other operational metrics that would support management decision-making.

Brief B: Property Leasing & Maintenance Platform

Scenario

A property management company oversees residential and commercial units across multiple buildings and locations. They handle the full leasing cycle: listing vacant units, processing tenant applications, managing active leases, and coordinating maintenance. Currently, tenants submit maintenance requests by phone or email, and there is no systematic way to track request status, assign maintenance staff, or monitor response times. The company is requesting a centralized platform for both leasing operations and maintenance management to smoothen overall operations.

Roles

Tenant, Property Manager, Maintenance Staff

Core Functional Areas

The following areas describe the core business needs your system must address. This list is not exhaustive; you are expected to analyze the scenario and identify additional requirements, features, and edge cases that would make the system complete and usable.

1. **Property and unit management.** The system manages buildings and their units. Each unit has attributes such as type, amenities, size, rent, and availability status. The property manager is responsible for managing the property and unit records.
2. **Tenant application and lease lifecycle.** Prospective tenants browse available units and submit lease applications. The lifecycle follows: Application → Screening → Approved/Rejected → Lease Active → Renewal / Termination. The system should ensure a unit cannot be leased to multiple tenants simultaneously and handle status transitions between tenants.

3. **Maintenance request lifecycle.** Tenants submit maintenance requests categorized by type and priority. The lifecycle follows: Submitted → Assigned → In Progress → Resolved → Closed. The property manager assigns requests to maintenance staff.
4. **Maintenance staff management.** Each maintenance staff member has a skill profile and an availability status. The system should support assigning appropriate staff to requests based on skill type.
5. **Payment tracking.** The system records lease payment installments. The property manager can record payments received and the system flags overdue balances.
6. **Notifications.** Tenants and maintenance staff receive in-system notifications when relevant events occur.

Public Maintenance Lookup

A public web page where a tenant can enter their **maintenance ticket number and registered phone number** (without logging in) to view the current status of their request and its history. The page must retrieve the details by calling a Web API using HttpClient. **This feature supports tenants who do not have an account or are unable to access it.**

Real-Time Maintenance Board

A live maintenance request board for staff that updates in real time when new requests are submitted or when a request's status/assignment changes

Reporting Application Guidance

The reporting application for this brief should provide the Property Manager with operational reports. Consider reports such as occupancy rates across buildings, maintenance backlog and average resolution times, overdue payment summaries, or any other operational metrics that would support management decision-making.

Brief C: Training & Certification Platform

Scenario

A professional training provider offers instructor-led courses across multiple subjects. They manage classrooms, instructors, course schedules, and student enrollments. Some courses are standalone, while others are part of certification tracks that require completing a set of courses to earn a certificate. Currently, enrollment is handled via emails and spreadsheets, making it difficult to track capacity, prevent scheduling conflicts, or verify certification status. The provider wants a web-based platform to manage the full training lifecycle from course scheduling through certification.

Roles

Trainee, Instructor, Training Coordinator

Core Functional Areas

The following areas describe the core business needs your system must address. This list is not exhaustive; you are expected to analyze the scenario and identify additional requirements, features, and edge cases that would make the system complete and usable.

1. **Course catalog management.** Courses should belong to categories or subjects and have descriptions, duration, and capacity limits. Some courses may have a single-level prerequisite (a trainee must complete Course A before enrolling in Course B).
2. **Instructor management.** Instructors have defined subject expertise areas and availability schedules. An instructor may teach multiple courses, but the system must prevent double booking.
3. **Classroom and resource management.** Rooms have capacity limits and may have specific equipment (projector, lab computers, etc.). The training coordinator is required to match rooms with course requirements.

4. **Course scheduling.** The training coordinator creates scheduled sessions by assigning an instructor, room, date, time, and capacity to a course. The system must validate against instructor and room availability before confirming the schedule.
5. **Enrollment lifecycle.** Trainees browse and enroll in available courses. The lifecycle follows: Enrolled → Confirmed → Attending → Completed / Dropped. Enrollment must validate prerequisites and capacity limits.
6. **Assessment recording.** After a course session ends, the instructor records a pass or fail result for each enrolled trainee. Passing a required course contributes toward certification progress.
7. **Certification tracking.** A certification track defines a set of required courses. When a trainee has passed all required courses, their status changes to eligible and a certificate can be generated through the system. The system should show trainees their progress toward certification.
8. **Payment tracking.** Each course has an enrollment fee. The training coordinator can record payments received (full or partial). The system tracks outstanding balances per trainee and flags overdue payments.
9. **Notifications.** Trainees and instructors receive in-system notifications when relevant events occur.

Public Certification Lookup

A public web page where a trainee or employer can enter a **trainee ID and a certificate reference number** (without logging in) to view the trainee's certification status and completed courses. The page must retrieve the details by calling a Web API using HttpClient. **This feature supports the verification of certifications by third parties such as employers.**

Real-Time Enrollment Counter

A live enrollment counter on course pages that updates in real time as trainees enroll, showing remaining available spots instantly.

Reporting Application Guidance

The reporting application for this brief should provide the Training Coordinator with operational reports. Consider reports such as enrollment statistics by course or category, instructor workload distribution, certification completion rates, revenue collected versus outstanding, or any other operational metrics that would support management decision-making.

4. Deliverables

Your final submission must include the following:

4.1 Solution & Source Code

A single Visual Studio solution containing all three projects (Web API, MVC Application, and Reporting Application). The solution must compile and run. **Your source code is expected to be delivered via a Git repository that will serve as your primary code submission.** The repository must be public and accessible for assessment. It should contain your complete solution alongside the commit history showing contributions from all team members. **The repository URL must be included in the project document.**

4.2 Database Scripts

- A SQL script that creates the database schema (all tables, constraints, relationships).
- A SQL script that seeds the database with test data, including user accounts for each role.
- If using separate databases for Identity and application data, submit both scripts.

4.3 Project Document

A document (PDF or Word) using a provided template that includes:

- **ERD:** Database design diagram (ERD) with a brief description of entities and justification for key design decisions and/or assumptions made.

- **Deployed URLs:** The public Azure URL(s) for your MVC application, Web API, and Reporting Application.
- **Demo credentials:** A table listing username and password for each role in your system (for both local and deployed versions).
- **API documentation:** A summary table of your API endpoints (route, HTTP method, purpose, authentication requirement).
- **NuGet packages:** A list of third-party NuGet packages used in the solution, with a brief description on the purpose of each.
- **System walkthrough:** A brief visual overview of the system showing the pages for each user role (with short captions). This is not a detailed feature documentation, but rather an overview of the system's scope and user interface.
- **Enhancements:** A brief description of any creative enhancements, additional features, or edge case handling beyond the baseline requirements (Include a screenshot for each enhancement).

5. Quality Standards

Beyond functional correctness, the work is assessed on the following quality dimensions:

5.1 Code Quality

- Code is well-structured, organized into appropriate classes and methods, and easy to read, following the MVC Architecture.
- Domain models and data access logic are cleanly organized (e.g., in clear folders or namespaces).
- Variables, methods, and classes have meaningful, descriptive names.
- Code is commented where necessary to explain non-obvious logic.
- Exception handling and input validation is implemented throughout.

- Database calls are minimized (non-redundant) and data is persisted efficiently.
- Views are strongly typed where applicable.

5.2 User Interface Quality

- The interface is clean, consistent, and intuitive across all pages.
- The web application is responsive and works on different screen sizes.
- Users receive appropriate feedback for their actions (success messages, error messages, loading states).
- Search, filter, and navigation features work correctly and efficiently.
- Role-based UI elements are correctly shown or hidden based on the logged-in user.

5.3 Security

- Authentication and authorization are correctly implemented (users cannot access what they should not).
- Passwords are not stored in plain text.
- API endpoints are secured with JWT and enforce role-based access.
- Input is validated on both client and server side.

5.4 Deployment

- The application is deployed using Azure App Service and accessible via the shared public URL.
- The database is hosted on Azure SQL Database.
- The deployed application is functional: users can register, log in, and use all system features.
- Connection strings and configuration are correctly set for the deployed environment.

6. Marking Criteria

The project is worth **30% of your course grade**. Your submission will be assessed across the following areas:

Assessment Area	Weight	What Is Assessed
Database & Data Layer	15%	Schema design, normalization, EF Core modeling, relationships, seeding, script quality
MVC Application – Functionality	25%	Feature completeness against brief requirements, business logic, lifecycle workflows, authentication & role-based access
MVC Application – Quality	10%	UI/UX consistency, responsiveness, code structure, validation, error handling
Web API	15%	RESTful endpoints (min. 5), JWT authentication, proper HTTP conventions, public tracking page consuming API via HttpClient, API documentation
Reporting Application	10%	Separate client consuming API via HttpClient with JWT, report views displaying operational data, correct technology implementation (MVC/Razor Pages/Blazor Server)
Real-Time Feature (SignalR)	10%	Working SignalR implementation addressing a suitable use case in the scenario
Deployment & Architecture	10%	Project structure, clean separation between projects, Azure deployment (App Service + Azure SQL), deployed application is functional, configuration management
Enhancements & Edge Cases	5%	Creative additions beyond baseline requirements, edge case handling.

Each area is scored on a **0–5 scale** using detailed descriptors. In addition to code review and live functionality testing (both local and deployed), the submitted project document will be referenced during assessment.

7. Group Work & Individual Accountability

7.1 Group Formation

- **Groups of 4–5 students.** Group formation will be coordinated with approval from the tutor.
- Each group selects one brief. A maximum of 2 groups per class section may choose the same brief.
- Each group member should take a lead role on at least one component or functional area, while contributing collaboratively across the project.

7.2 Git Requirement

All groups are expected to use a Git-based version control system (GitHub, GitLab, or Azure DevOps) throughout the project. Your repository is the primary submission method for your source code and must be public. Your commit history serves as a record of individual contributions and will be referenced if there are disputes about participation. Commits should be made by individual team members using their own accounts.

The standard procedures for plagiarism and non-participation apply. This may include separation from the group, failing the project component, or other actions per the Bahrain Polytechnic policy.

8. Individual Reflection

Each student must submit an individual reflection that is linked to the project assessment. The reflection is worth **10% of your course grade** and is assessed separately from the group project mark. It consists of two equally weighted parts:

8.1 - Part 1: Written Reflection (5%)

Submit a document (PDF or Word) that evidences your individual contribution to the group project. Your written reflection must include the following:

Contribution Table. A table listing each feature or component you personally worked on, a description of what you did, and a reference to the relevant Git commit(s). This should clearly show the scope and nature of your individual work.

Technical Decisions. Describe 2–3 technical decisions you made or were significantly involved in during the project. For each decision, explain what the decision was, what alternatives were considered, and why the chosen approach was selected.

Technical Challenge. Describe one significant technical challenge you faced during the project and how it was resolved. Reference specific code, errors, or architectural problems encountered. Explain your debugging or problem-solving process.

Your reflection should include screenshots and code snippets with explanations that demonstrate your understanding of the work you contributed. Claims made in your reflection should be consistent with your Git commit history and should not contradict the contributions claimed by other members of your group. Inconsistencies between group members' reflections may result in mark reductions.

8.2 Part 2: In-Person Observation (5%)

Each student will attend a short individual session with their tutor. This may take place as a scheduled session or during in-class project review periods. During this session, the tutor will:

- Ask you to explain and walk through the parts of the project you have contributed to and/or mentioned in the written reflection
- Ask follow-up questions about your code, your design decisions, and the technical challenge you described.

Students who cannot adequately explain their claimed contributions will receive a reduced mark for this component.

Reflection Assessment Summary

Component	Weight	Key Assessment Criteria
Written Reflection	5%	Contribution evidence with Git references, quality of technical decision analysis, depth of challenge description, overall clarity and professionalism
In-Person Observation	5%	Ability to explain own code, understanding of design decisions, depth of technical knowledge